Relatório – Atividade 4
Random Neurons vs. Random Forest

Caique Frederico Pereira de Moura
Aprendizado Computacional e Reconhecimento de Padrões

30 de abril de 2026

1 Objetivo

O objetivo desta atividade foi implementar o método *Random Neurons*, uma versão simplificada de *Random Forest*, e avaliá-lo em diferentes configurações de parâmetros L e M nos conjuntos *Iris* e *Wine*. Em seguida, comparou-se o desempenho (percentual de acerto) com um *Random Forest* configurado com L árvores.

2 Método: Random Neurons

Considere um conjunto de treinamento $D = \{(\mathbf{x}_i, y_i) \in \mathbb{R}^n \times \{1, 2, \ldots, c\} : i = 1, \ldots, m\}$. O *Random Neurons* foi implementado seguindo o roteiro proposto:

1. **Bootstrap (bagging).** Para um dado $L \in \mathbb{N}$, foram geradas L réplicas D_ℓ de D por amostragem com reposição (bootstrap), com $\ell = 1, \ldots, L$.

2. **Subespaço aleatório de atributos.** Para cada réplica D_ℓ, sorteou-se um inteiro $k \sim \mathcal{U}(\{1, \ldots, n\})$ e selecionou-se, sem reposição, um subconjunto de k atributos. Os ındices sorteados foram armazenados para uso posterior.

3. **Treinamento de classificadores base.** Para cada $\ell = 1, \ldots, L$, foi treinado um Perceptron de Bolso (*Pocket Perceptron*) limitado a M iterações, utilizando apenas o subconjunto de atributos sorteado para aquele modelo.

4. **Multiclasse via OVR.** Como Iris e Wine são multiclasse, adotou-se *one-vs-rest* (OVR): para cada classe c, ajusta-se um classificador binário que separa c contra as demais.

5. **Votação por maioria.** Para classificar um padrão $\mathbf{x}$, obteve-se a predição de cada um dos L modelos (com seus respectivos subconjuntos de atributos) e escolheu-se como saída a classe mais frequente.

Em termos conceituais, o *Random Neurons* combina *bagging* com *random subspace method*, trocando as árvores do Random Forest por classificadores lineares (Perceptron de Bolso), o que tende a aumentar o viés do modelo quando a fronteira de decisão é não linear.

3 Protocolo experimental

Os experimentos foram realizados conforme a implementação em Python:

- **Bases de dados:** *Iris* e *Wine* (`sklearn.datasets`).
- **Divisão treino-teste:** 70%/30% com `random_state=42`.
- **Pré-processamento:** padronização (`StandardScaler`) ajustada no treino e aplicada no teste.
- **Parâmetros avaliados (Random Neurons):** $L \in \{5, 10, 20\}$ e $M \in \{50, 100, 200\}$.
- **Baseline (Random Forest):** `n_estimators = L`, `max_depth=None`, `min_samples_split=2`, `random_state=42`.
- **Métrica:** acurácia no conjunto de teste.

Pré-processamento e reprodutibilidade. Os dados foram normalizados utilizando *StandardScaler*, etapa essencial para o bom desempenho de modelos lineares como o Perceptron. Além disso, foi fixada uma semente pseudoaleatória (`random_state = 42`) para garantir reprodutibilidade dos experimentos.

Configuração do Random Forest. O Random Forest foi configurado com $n_estimators = L$, $max_depth = None$ e $min_samples_split = 2$. A escolha de $max_depth = None$ permite o crescimento completo das árvores, maximizando sua capacidade de modelagem e servindo como um comparativo forte em relação ao método proposto.

4 Resultados

As Tabelas 1 e 2 apresentam os percentuais de acerto observados para o *Random Neurons*. Para o *Random Forest*, a acurácia obtida para cada L (com L árvores) está indicada na última coluna.

Tabela 1: Iris: acurácia (%) para diferentes L e M.

Configuração	$M = 50$	$M = 100$	$M = 200$	**RF (L árvores)**
$L = 5$	82.22	91.11	77.78	100.00
$L = 10$	86.67	88.89	75.56	100.00
$L = 20$	88.89	84.44	75.56	100.00

Tabela 2: Wine: acurácia (%) para diferentes L e M.

Configuração	$M = 50$	$M = 100$	$M = 200$	**RF (L árvores)**
$L = 5$	94.44	100.00	96.30	92.59
$L = 10$	96.30	100.00	98.15	92.59
$L = 20$	100.00	100.00	100.00	96.30

As Figuras 1 e 2 sintetizam os resultados, mostrando como a acurácia varia com M e comparando com a linha de referência do *Random Forest* para cada L.

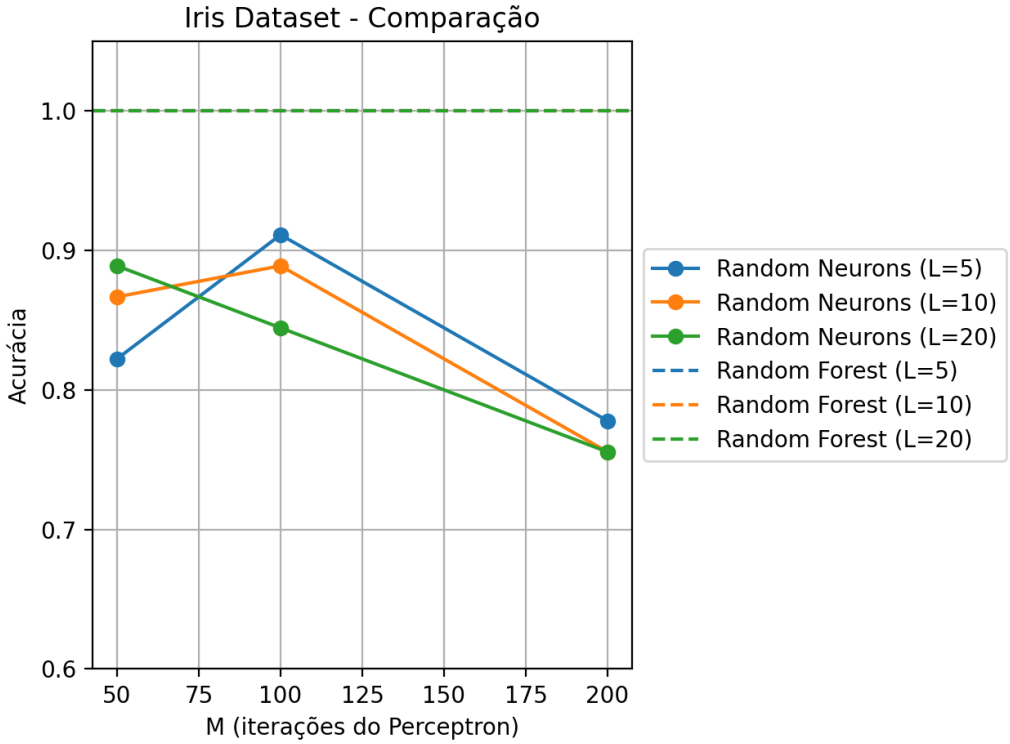

Figura 1: Iris: comparação entre Random Neurons e Random Forest.

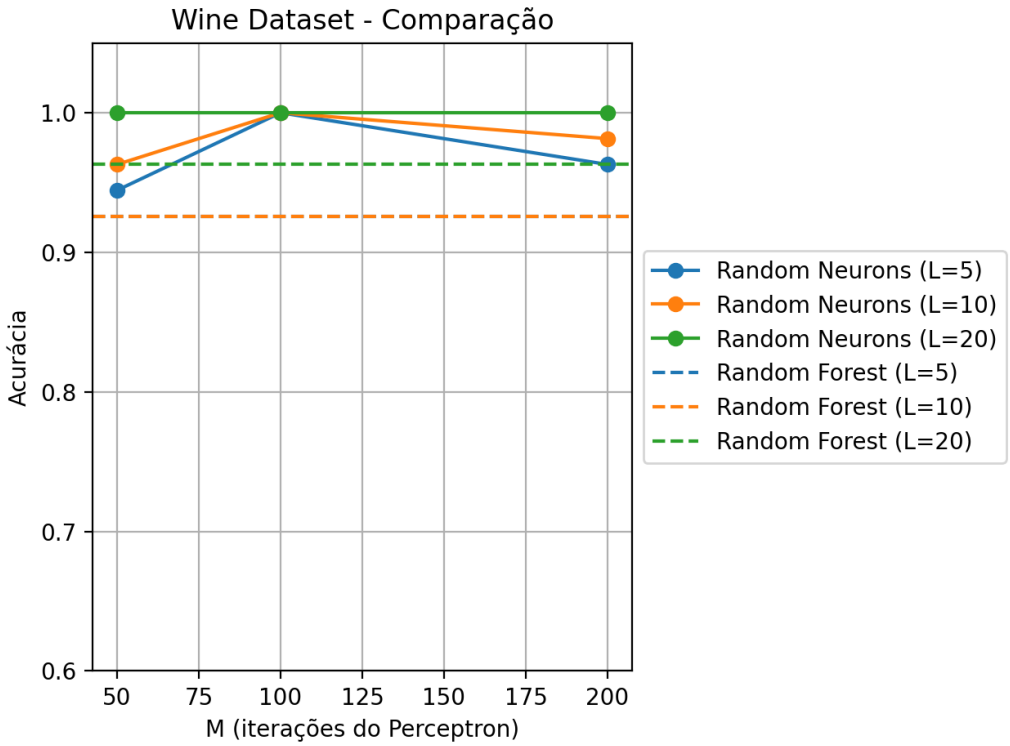

Figura 2: Wine: comparação entre Random Neurons e Random Forest.

5 Discussão

No *Iris*, o Random Forest obteve 100% de acurácia para os valores de L avaliados, enquanto o Random Neurons variou de aproximadamente 75.56% a 91.11%. Esse resultado é consistente com o fato de que o Random Neurons utiliza apenas classificadores lineares (Perceptron), o que impõe maior viés e pode não capturar fronteiras de decisão não lineares. Além disso, o sorteio aleatório de atributos pode produzir subconjuntos pouco informativos, degradando o desempenho.

No *Wine*, o comportamento foi distinto: o Random Neurons atingiu 100% de acurácia em diversas configurações (por exemplo, $M = 100$ para $L = 5$, $L = 10$ e $L = 20$), superando o Random Forest em algumas delas. Isso sugere que, no Wine, existem subespaços de atributos em que a separabilidade linear é alta e o ensemble (via bagging + subespaços) consegue compensar a instabilidade de um classificador linear simples.

Quanto ao parâmetro M, os resultados indicaram que aumentar o número máximo de iterações do Perceptron nem sempre melhora a acurácia: em alguns casos, $M = 200$ não trouxe ganhos e pode até reduzir o desempenho, possivelmente por oscilações no ajuste e sensibilidade à amostra bootstrap e ao subespaço sorteado.

6 Conclusão

Os experimentos mostraram que o Random Forest é, em geral, mais robusto e confiável (principalmente no Iris), enquanto o Random Neurons pode ser competitivo quando há subespaços linearmente separáveis (como observado no Wine). Assim, o Random Neurons é uma alternativa interessante do ponto de vista didático para discutir bagging, seleção aleatória de atributos e votação, mas não substitui métodos mais expressivos em problemas com forte não linearidade.